

RAPPORT DE STAGE

Deuxième année — BTS SIO SLAM

(Solutions Logicielles et Applications Métiers)

Développement Full Stack d'une plateforme de gestion interne — Spring Boot & Angular

Stage effectué chez FenziConnect

Stagiaire : ALI YOUSOUF Facridine

Tuteur entreprise : Mme Melissa FENZI

Période du stage : Du 08 janvier 2026 au 20 février 2026 (6 semaines)

Établissement : ESIC — BTS SIO SLAM 2e année

Année universitaire 2025 — 2026

Remerciements

Avant d'entrer dans le vif du sujet, je tiens à exprimer ma sincère reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de ce stage de deuxième année et à la rédaction de ce rapport. Cette expérience professionnelle de six semaines au sein de la société FenziConnect a constitué une étape déterminante de mon parcours en BTS SIO option SLAM, tant sur le plan technique qu'humain.

Je remercie en premier lieu Madame Melissa FENZI, dirigeante de FenziConnect, pour la confiance qu'elle m'a accordée en m'intégrant à son équipe et en me confiant des missions de développement à fort enjeu opérationnel. Sa disponibilité, la pertinence de ses retours techniques ainsi que la liberté qu'elle m'a laissée dans la mise en œuvre des solutions ont profondément enrichi ma vision du métier de développeur Full Stack.

J'adresse également mes remerciements à l'ensemble de l'équipe technique de FenziConnect pour son accueil chaleureux et son esprit collaboratif. La méthodologie agile mise en place, l'usage rigoureux de Jira et de GitLab pour la gestion des sprints et le partage quotidien d'expériences lors des points de synchronisation m'ont permis de progresser rapidement dans un environnement professionnel exigeant.

Mes remerciements vont par ailleurs à toute l'équipe pédagogique de l'ESIC, et plus particulièrement à mes professeurs du BTS SIO SLAM, qui ont su, au cours de ces deux années, me transmettre les fondamentaux indispensables à ce type de mission : modélisation UML, conception orientée objet, bases de données relationnelles, programmation web et bonnes pratiques de génie logiciel. Sans cette base théorique solide, l'adaptation rapide au stack technique de l'entreprise n'aurait pas été possible.

Enfin, je remercie ma famille et mes proches pour leur soutien constant tout au long de ce stage, ainsi que mes camarades de promotion, dont les échanges autour de nos retours d'expériences respectifs ont nourri une réflexion comparative très formatrice.

Sommaire

Remerciements	2
Sommaire	3
I. Introduction	4
1. Le BTS SIO SLAM à l'ESIC	4
2. Objectifs du stage de 2e année	5
II. Présentation de l'entreprise FenziConnect	6
1. Identité et activité	6
2. Organisation et méthodologie de travail	7
3. Environnement technique et outils	7
III. Contexte et intégration au projet	8
1. Accueil et présentation du projet interne	8
2. Mise en place de l'environnement de développement	8
3. Posture professionnelle adoptée pendant le stage	9
IV. Mission 1 — Module « Évaluation Stagiaire » (FC-23)	10
1. Expression du besoin et modélisation	10
2. Implémentation côté backend (Spring Boot)	11
3. Implémentation côté frontend (Angular)	13
4. Tests, validation et flux d'enregistrement	14
V. Mission 2 — Dynamisation et gestion des Établissements (FC-24)	16
1. Contexte et expression du besoin	16
2. Conception SQL et création de la table « etablissement »	17
3. Évolution du backend Spring Boot	18
4. Refonte du frontend : tri, historique et UX	19

5. Modernisation des Modals et styles SCSS dynamiques	20
VI. Analyse comparative CACIB / FenziConnect	—
VII. Conclusion et perspectives professionnelles	—

I. Introduction

1. Le BTS SIO SLAM à l'ESIC

Le Brevet de Technicien Supérieur Services Informatiques aux Organisations (BTS SIO), option Solutions Logicielles et Applications Métiers (SLAM), est un diplôme de niveau 5 qui forme en deux ans des techniciens supérieurs capables d'analyser, de concevoir, de développer et de maintenir des applications informatiques au sein d'entreprises de tous secteurs. Cette formation, dispensée à l'ESIC, repose sur un équilibre rigoureux entre enseignements théoriques (algorithmique, modélisation, programmation orientée objet, bases de données relationnelles, cybersécurité) et pratique professionnelle, matérialisée notamment par deux stages obligatoires.

Le premier stage, effectué en fin de première année, vise généralement la découverte du milieu professionnel et l'appropriation d'un environnement de production. Le second stage, dont le présent rapport rend compte, intervient en deuxième année et marque une étape charnière : il s'agit désormais de prendre en charge des développements concrets, d'assumer une part d'autonomie sur des fonctionnalités livrables et d'éprouver, dans des conditions réelles, les compétences acquises en cours.

Ce stage de fin de cursus est également un moment privilégié pour confirmer ou réorienter son projet professionnel. Pour ma part, ayant déjà effectué mon premier stage en première année au sein du Crédit Agricole Corporate and Investment Bank (CACIB), grand groupe bancaire international, j'ai souhaité, pour cette deuxième expérience, intégrer une structure plus restreinte afin de toucher à un spectre fonctionnel plus large et de me rapprocher du cœur du métier de développeur. Le choix de FenziConnect, auto-entreprise spécialisée dans les solutions numériques sur mesure, répondait précisément à cette attente.

2. Objectifs du stage de 2e année

Le stage effectué chez FenziConnect du 8 janvier au 20 février 2026 poursuivait plusieurs objectifs, à la fois techniques, méthodologiques et personnels.

Objectifs techniques

- Maîtriser un stack Full Stack moderne articulé autour de Spring Boot pour la couche serveur et Angular (TypeScript) pour la couche cliente.
- Manipuler une base de données MySQL en environnement de production : conception du modèle logique, écriture de scripts SQL, gestion des contraintes d'intégrité référentielle.
- Implémenter de nouveaux endpoints REST sécurisés (méthodes HTTP PUT, GET, POST, DELETE) et les consommer côté client via les services Angular et le module HttpClient.
- Améliorer l'expérience utilisateur d'une application existante en modernisant les composants graphiques (Modals, alertes SweetAlert2, styles SCSS dynamiques).

- Comprendre et faire évoluer une configuration de sécurité applicative (filtres, rôles, autorisations) au sein d'un fichier SecurityConfig.java.

Objectifs méthodologiques

- M'intégrer dans une équipe qui fonctionne en mode agile hybride (Scrum allégé), avec rituels de planification, points quotidiens et démonstrations de fin de sprint.
- Utiliser quotidiennement les outils professionnels de gestion de projet et de versionnement : Trello, Jira, GitLab et GitHub.
- Respecter une chaîne de livraison rigoureuse : branche dédiée par ticket, revue de code, tests Postman, fusion sur la branche de développement.

Objectifs personnels

- Confirmer mon orientation vers le développement Full Stack en m'éprouvant sur une stack très différente de celle pratiquée pendant le premier stage.
- Développer mon autonomie dans la prise en main d'un code existant, qualité essentielle attendue d'un développeur en poste.
- Construire une vision comparative entre une grande structure cloisonnée (CACIB) et une petite structure polyvalente (FenziConnect), afin d'éclairer mes choix de poursuite d'études et de carrière.

II. Présentation de l'entreprise FenziConnect

1. Identité et activité

FenziConnect est une auto-entreprise française spécialisée dans la conception et le développement de solutions numériques sur mesure à destination des TPE, PME et organismes de formation. Fondée et dirigée par Madame Melissa FENZI, elle a son siège dans le 15^e arrondissement de Paris et opère selon un modèle hybride mêlant présentiel et télétravail. Ce positionnement, à la croisée de l'expertise technique et du service de proximité, lui permet de proposer à ses clients un accompagnement complet allant de l'expression du besoin jusqu'au déploiement et à la maintenance applicative.

Le cœur de métier de FenziConnect s'articule autour de trois grands axes : le développement web Full Stack (applications de gestion, plateformes SaaS, espaces clients), la sécurité applicative (gestion des authentifications, contrôle d'accès, durcissement des configurations Spring) et la modernisation d'applications existantes (reprise de code historique, refonte UX, migration de bases). Cette polyvalence est caractéristique des structures de petite taille, où chaque collaborateur — y compris un stagiaire — est amené à intervenir sur l'ensemble de la chaîne de production logicielle.

Carte d'identité de l'entreprise

Information	Détail
Raison sociale	FenziConnect
Forme juridique	Auto-entreprise
Dirigeante	Mme Melissa FENZI
Localisation	Paris 15 ^e — France
Secteur d'activité	Développement web, SaaS, sécurité applicative
Mode de travail	Hybride (présentiel / télétravail)
Effectif	Petite structure — équipe technique restreinte
Clients types	TPE, PME, organismes de formation

2. Organisation et méthodologie de travail

Compte tenu de sa taille, FenziConnect s'appuie sur une organisation horizontale et une méthodologie agile adaptée. Les développements sont découpés en tickets, chacun identifié par un préfixe projet (par exemple FC-23 ou FC-24 pour les missions qui m'ont été confiées) et associé à un responsable, à une priorité et à un sprint. L'organisation des sprints s'inspire de Scrum sans toutefois en reproduire l'intégralité des cérémonies : on retrouve une planification de début de sprint, des points quotidiens courts, une revue de fin de sprint et une rétrospective ramassée. Cette légèreté est rendue possible par le faible nombre d'intervenants et favorise une grande réactivité face aux retours clients.

Le suivi opérationnel s'effectue à travers Trello pour la vision synthétique des cartes (« To Do », « In Progress », « Review », « Done ») et Jira pour la traçabilité fine des tickets, des estimations et des liens entre user stories et sous-tâches. La communication informelle passe par un canal de discussion dédié, sur lequel sont également relayées les notifications GitLab et GitHub.

3. Environnement technique et outils

L'environnement technique de FenziConnect est volontairement homogène afin de capitaliser sur des compétences transversales d'un projet à l'autre. Le tableau suivant synthétise les principales technologies rencontrées pendant le stage.

Couche	Technologies / Outils
Backend	Java 17, Spring Boot 3, Spring Security, Spring Data JPA, Hibernate
Frontend	Angular 16+, TypeScript, RxJS, SCSS, SweetAlert2
Base de données	MySQL 8 (relations 1-n, contraintes d'intégrité référentielle)
API & tests	REST (JSON), Postman pour les tests d'endpoints
Versionnement	Git, GitLab, GitHub (branches par ticket, merge requests)
Gestion de projet	Jira, Trello
Environnement local	Visual Studio Code, JDK 17, Node.js / npm

III. Contexte et intégration au projet

1. Accueil et présentation du projet interne

Le premier jour de stage, lundi 8 janvier 2026, a été consacré à mon intégration dans la structure. Madame FENZI m’a accueilli à l’agence et m’a présenté l’équipe ainsi que les locaux. Une présentation détaillée du projet sur lequel j’allais intervenir m’a ensuite été faite : il s’agit d’une plateforme interne de gestion développée par FenziConnect pour le compte d’organismes de formation, et destinée à centraliser le suivi des stagiaires, des établissements partenaires et des évaluations associées. L’application existait déjà dans une première version fonctionnelle, et mes missions consistaient à l’ étoffer avec deux modules métiers (FC-23 et FC-24) et à en moderniser l’expérience utilisateur.

Cette journée d’accueil s’est poursuivie par une lecture commune du code source du projet, organisée autour d’un poste partagé. Cette approche — souvent qualifiée de « pair reading » — a permis de comprendre rapidement la structure du dépôt (séparation claire entre frontend Angular et backend Spring Boot), les conventions de nommage internes, ainsi que les principaux points d’attention en matière de sécurité (utilisation de SecurityConfig.java pour la configuration des filtres et des rôles).

2. Mise en place de l’environnement de développement

Le second jour a été consacré à la mise en place de mon environnement de développement local, étape indispensable avant toute contribution. Cette installation, bien que classique, m’a confronté à plusieurs particularités liées au stack utilisé chez FenziConnect.

L’IDE retenu est Visual Studio Code, complété par les extensions « Spring Boot Extension Pack », « Angular Language Service », « Prettier » et « GitLens ». Le JDK 17 a été installé conformément à la version cible du projet, ainsi que Node.js et npm pour la gestion des dépendances Angular. La base de données MySQL a été déployée localement, peuplée d’un jeu de données de test fourni par l’équipe, afin de pouvoir tester l’application de bout en bout sans dépendre de l’environnement de recette. Enfin, Postman a été configuré avec une collection partagée contenant les principaux endpoints REST de l’application, ce qui facilite la vérification de chaque modification côté serveur.

L’installation s’est conclue par un premier clonage du dépôt GitLab sur la branche develop, suivi d’un build complet : compilation Maven côté backend, puis exécution de la commande npm install et lancement du serveur de développement Angular via ng serve. Cette première exécution sans modification a permis de valider le bon fonctionnement de la chaîne d’outillage et de constituer une base saine pour les développements à venir.

3. Posture professionnelle adoptée pendant le stage

Au-delà des aspects strictement techniques, j'ai très tôt cherché à adopter une posture professionnelle alignée sur les attentes d'une petite structure. La taille réduite de l'équipe rend chaque intervention immédiatement visible, ce qui impose une rigueur particulière dans la qualité du code livré, dans la rédaction des messages de commit et dans la traçabilité des décisions prises. Pour y répondre, j'ai mis en place une routine quotidienne consistant à ouvrir une branche dédiée à chaque ticket, à effectuer des commits atomiques et à rédiger systématiquement une description claire des merge requests, de manière à faciliter la relecture par Madame FENZI.

J'ai également veillé à formaliser, dans un document de suivi personnel, les questions techniques que je me posais avant les points quotidiens, afin de tirer le meilleur parti de ces moments d'échange et de ne pas freiner la progression du sprint. Cette discipline individuelle, complétée par une lecture régulière de la documentation officielle de Spring Boot, d'Angular et de SweetAlert2, m'a permis de monter en autonomie au fil des semaines, et constitue à mes yeux l'un des acquis les plus durables de cette expérience chez FenziConnect.

IV. Mission 1 — Module « Évaluation Stagiaire » (FC-23)

1. Expression du besoin et modélisation

La première mission qui m’a été confiée, identifiée par le ticket FC-23, portait sur la création d’un module dédié à l’évaluation des stagiaires. Le besoin exprimé par le client était simple en apparence mais structurant pour la plateforme : permettre à un administrateur de saisir, pour chaque stagiaire, une note chiffrée accompagnée d’un commentaire qualitatif, et conserver un historique horodaté de ces évaluations afin d’assurer un suivi pédagogique pertinent.

Avant tout développement, une phase de modélisation a été conduite afin de positionner correctement la nouvelle entité Note dans le modèle métier existant. Le diagramme de classes UML ci-dessous, réalisé en amont du codage, restitue les associations retenues entre les entités Etablissement, Stagiaire, Evaluation et Note.

Diagramme de Classes UML - Application FenziConnect

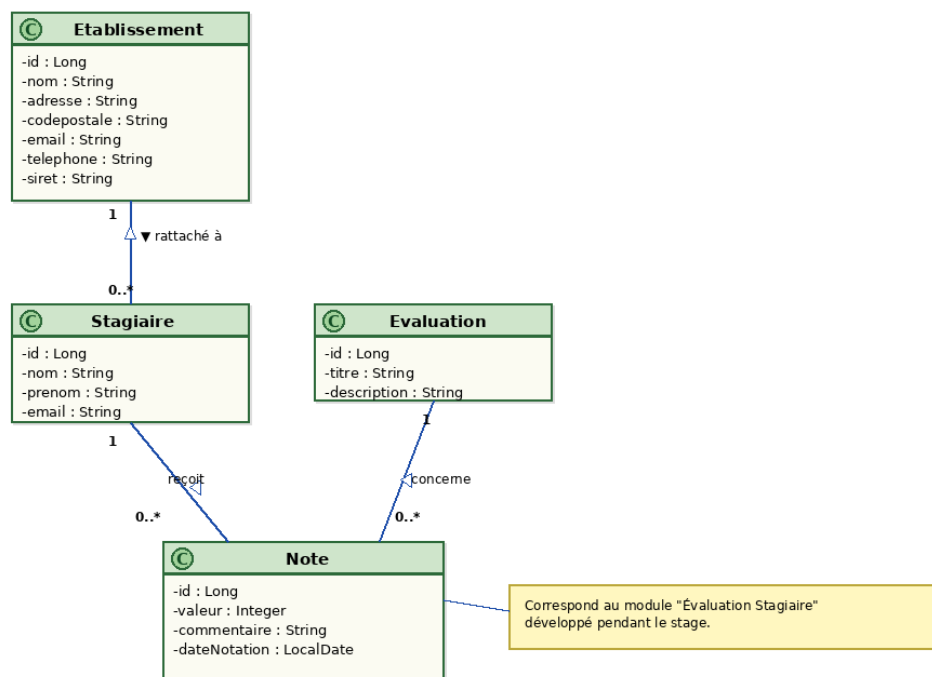


Figure 1 — Diagramme de classes UML de l'application FenziConnect.

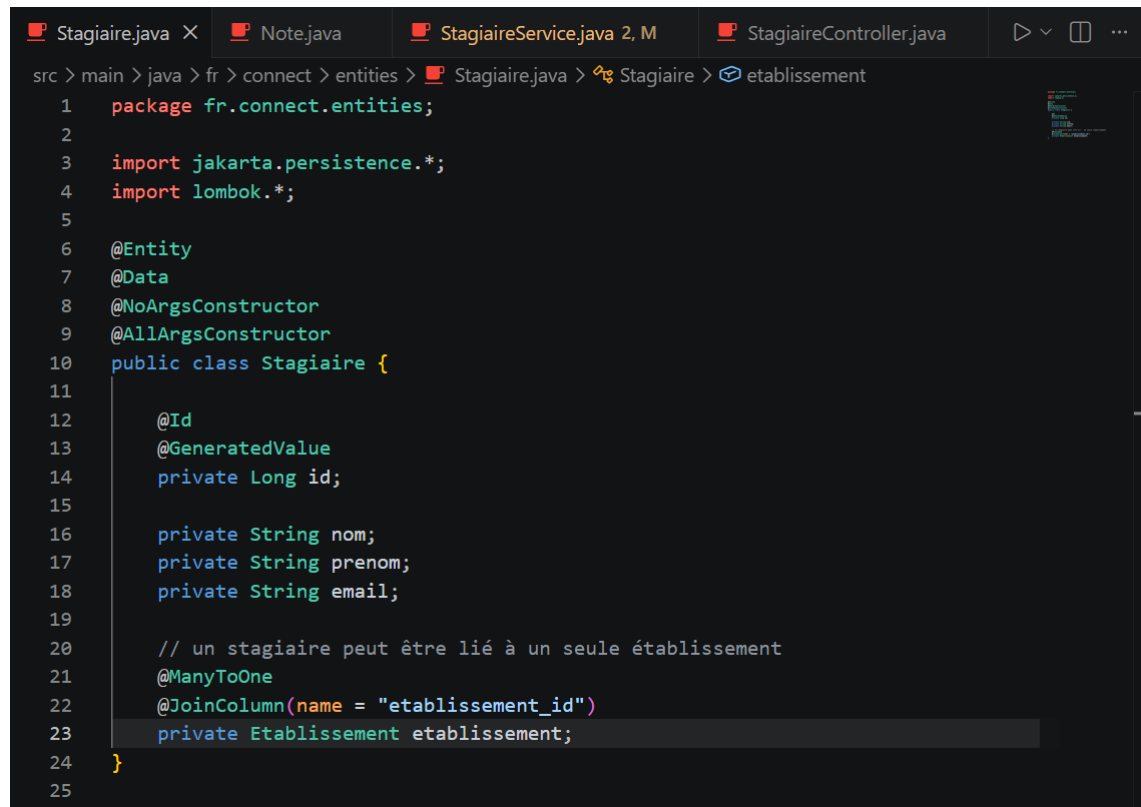
2. Implémentation côté backend (Spring Boot)

L’implémentation côté serveur a constitué la première étape concrète de la mission FC-23. Pour respecter l’architecture en couches déjà en place dans le projet, le développement a été organisé selon la séquence classique : enrichissement du modèle de données, exposition d’une opération de mise à jour dans le contrôleur, propagation de la logique métier dans le service

applicatif et persistance via Spring Data JPA. L'objectif était de proposer un endpoint REST robuste, testable indépendamment du frontend, et conforme aux conventions internes (URI versionnée, codes de retour HTTP explicites, payload JSON typé).

Évolution du modèle Stagiaire

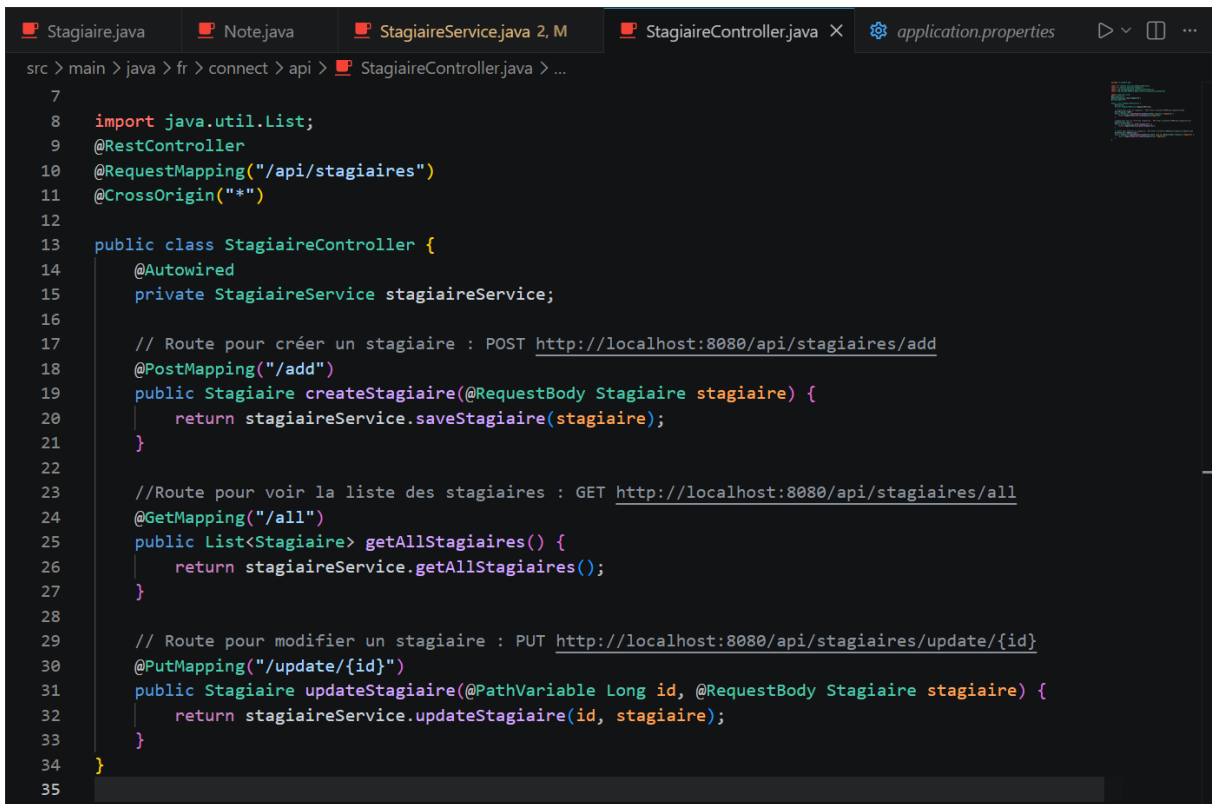
Pour répondre au besoin sans remettre en cause le modèle initial, deux attributs ont été ajoutés à l'entité Stagiaire : une note (de type Integer) et un commentaire (de type String). Ces attributs ont été annotés de manière à être pris en charge automatiquement par Hibernate lors de la génération du schéma. La date de notation a été introduite via un champ LocalDate alimenté côté serveur au moment de la mise à jour, afin d'éviter toute désynchronisation entre l'horloge du client et celle du serveur.



```
src > main > java > fr > connect > entities > Stagiaire.java > Stagiaire > etablissement
1  package fr.connect.entities;
2
3  import jakarta.persistence.*;
4  import lombok.*;
5
6  @Entity
7  @Data
8  @NoArgsConstructor
9  @AllArgsConstructor
10 public class Stagiaire {
11
12     @Id
13     @GeneratedValue
14     private Long id;
15
16     private String nom;
17     private String prenom;
18     private String email;
19
20     // un stagiaire peut être lié à un seule établissement
21     @ManyToOne
22     @JoinColumn(name = "etablissement_id")
23     private Etablissement etablissement;
24 }
25
```

Endpoint PUT dans StagiaireController

L'opération de mise à jour a été exposée via un endpoint REST PUT `/api/stagiaires/update/{id}`. Le choix du verbe HTTP PUT découle directement de la sémantique métier : il s'agit d'une mise à jour ciblée d'une ressource identifiée par son id. La méthode du contrôleur délègue la logique au service, retourne un statut 200 OK en cas de succès et un 404 si le stagiaire est introuvable, ce qui permet au frontend de réagir précisément à chaque cas.



```
7
8 import java.util.List;
9 @RestController
10 @RequestMapping("/api/stagiaires")
11 @CrossOrigin("*")
12
13 public class StagiaireController {
14     @Autowired
15     private StagiaireService stagiaireService;
16
17     // Route pour créer un stagiaire : POST http://localhost:8080/api/stagiaires/add
18     @PostMapping("/add")
19     public Stagiaire createStagiaire(@RequestBody Stagiaire stagiaire) {
20         return stagiaireService.saveStagiaire(stagiaire);
21     }
22
23     //Route pour voir la liste des stagiaires : GET http://localhost:8080/api/stagiaires/all
24     @GetMapping("/all")
25     public List<Stagiaire> getAllStagiaires() {
26         return stagiaireService.getAllStagiaires();
27     }
28
29     // Route pour modifier un stagiaire : PUT http://localhost:8080/api/stagiaires/update/{id}
30     @PutMapping("/update/{id}")
31     public Stagiaire updateStagiaire(@PathVariable Long id, @RequestBody Stagiaire stagiaire) {
32         return stagiaireService.updateStagiaire(id, stagiaire);
33     }
34 }
35
```

Figure 2 — Capture du fichier StagiaireController.java (méthode update).

3. Implémentation côté frontend (Angular)

Côté client, l'objectif était double : offrir une saisie ergonomique de la note et du commentaire à l'administrateur, et fournir un retour visuel immédiat indiquant le succès ou l'échec de l'opération. Le projet utilisait déjà la bibliothèque SweetAlert2 pour les notifications, choix retenu pour rester homogène avec les autres écrans. La logique de communication réseau a été encapsulée dans un service Angular dédié, de manière à isoler les composants graphiques des appels HTTP et faciliter d'éventuels tests unitaires futurs.

Formulaire de saisie

Le formulaire a été construit à l'aide de Reactive Forms, ce qui permet de centraliser la déclaration des contrôles et de leurs validateurs (note comprise entre 0 et 20, commentaire non vide, longueur maximale). Le composant n'est rendu actif qu'après validation : tant que les champs ne sont pas correctement renseignés, le bouton « Enregistrer » reste désactivé, ce qui constitue une première barrière côté client avant l'appel au serveur.

Nouveau Formulaire

Stagiaire : Facridine Ali youssouf

Module / Examen : Choisir un module...

Note obtenue : 20 / 20

Commentaire professionnel :

Observations sur le travail du stagiaire...

Enregistrer l'évaluation

Historique Récent

- 15 JAN 2026
"Excellent travail..."
Par : KOUEININ Prisca
19/20
- 02 JAN 2026
"Bon debut mais..."
Par : KOUEININ Prisca
12/20
- 22 JAN 2026
"Bon travail..."
Par : KOUEININ Prisca
15/20

Consulter tout l'historique

Figure 3 — Capture d'écran du formulaire d'évaluation (composant Angular).

Service Angular et SweetAlert2

Le service `StagiaireService` expose une méthode `updateStagiaire(id, payload)` qui retourne un `Observable`, consommé par le composant via un `subscribe`.

```
@Injectable({
  providedIn: 'root'
})
export class StagiaireService {

  private apiUrl = 'http://localhost:8080/api/stagiaires/all'; // URL de l'API backend

  constructor(private http: HttpClient) {}

  // Récupérer tous les stagiaires
  getStagiaires(): Observable<Stagiaire[]> {
    return this.http.get<Stagiaire[]>(this.apiUrl);
  }
}
```

4. Tests, validation et flux d'enregistrement

La phase de validation a combiné des tests manuels via Postman pour l'endpoint REST et une validation visuelle de bout en bout depuis l'interface Angular. Cette double approche permet de différencier les anomalies issues du serveur de celles introduites côté client, et d'itérer plus efficacement.

Tests Postman

Une requête PUT a été construite dans Postman avec un corps JSON contenant les champs `note`, `commentaire` et `identifiant cible`. Le serveur retourne un statut 200 OK accompagné de l'objet `Stagiaire` mis à jour. Les tests négatifs (note hors plage 0-20, identifiant inexistant, payload mal formé) ont également été couverts pour valider la robustesse du contrôleur.

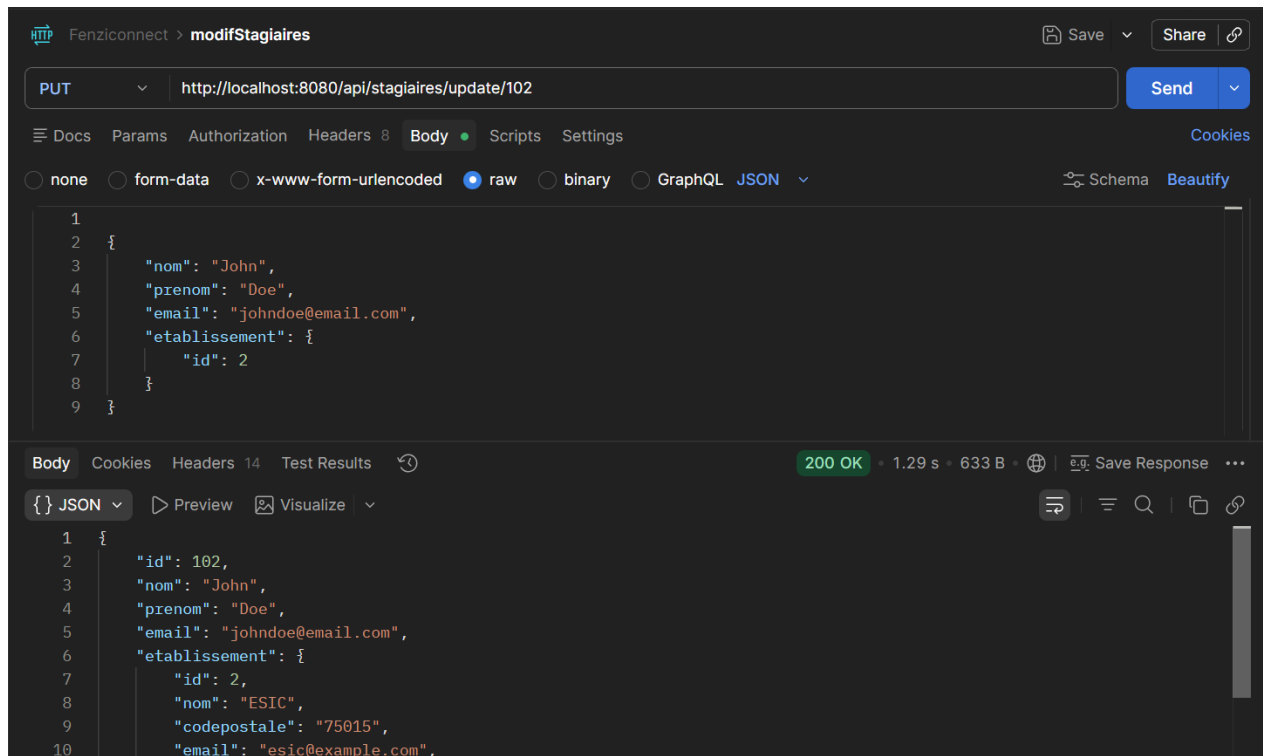


Figure 4 — Test Postman : PUT /api/stagiaires/update/12 → 200 OK.

Flux global d'enregistrement

Le diagramme de séquence ci-après synthétise les échanges entre les quatre acteurs impliqués lors d'un enregistrement d'évaluation : l'administrateur, l'interface Angular, le contrôleur Spring Boot et la base MySQL. Il met en évidence les étapes de validation côté client, le passage de la requête HTTP, la vérification des permissions par SecurityConfig et la confirmation SQL avant le rafraîchissement de l'historique réel à l'écran.

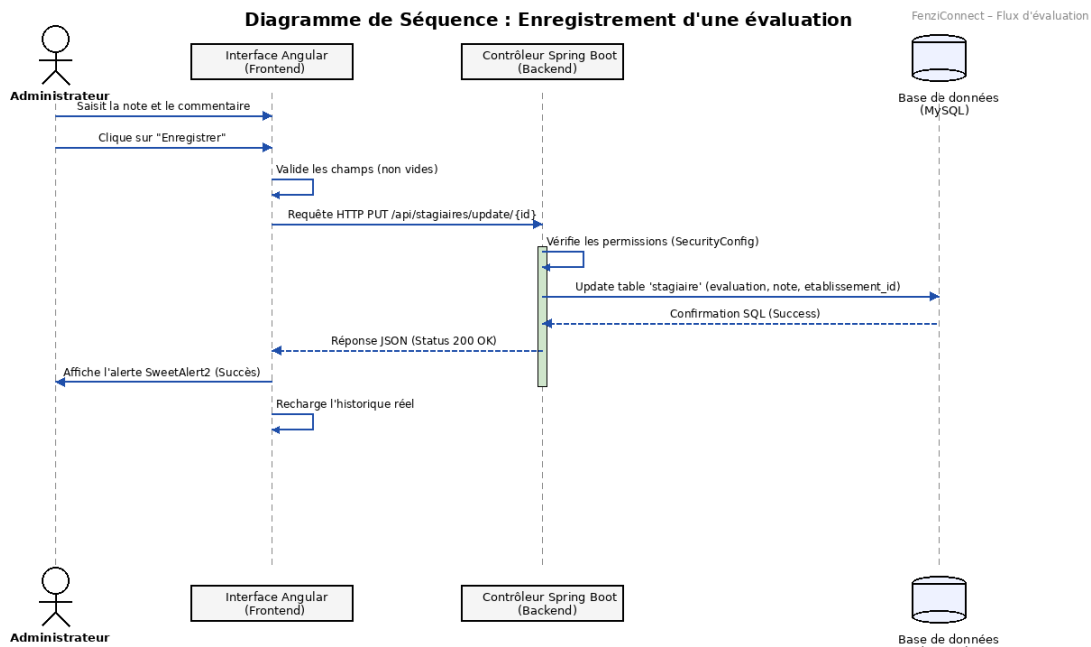


Figure 5 — Diagramme de séquence : enregistrement d'une évaluation.

Bilan de la mission FC-23

Cette première mission m'a permis de réaliser un développement Full Stack complet de bout en bout, depuis la modélisation jusqu'à la validation par tests. Elle a également constitué un excellent terrain d'apprentissage de Spring Data JPA et d'Angular Reactive Forms, deux briques que je n'avais jusque-là pratiquées que dans un cadre pédagogique. Le ticket FC-23 a été clôturé sur Jira après validation par Madame FENZI.

SUIVI & ÉVALUATION DES STAGIAIRES

Description
Mettre en place le suivi pédagogique.

Backend – Spring Boot

- Ajouter à **Stagiaire** :
 - evaluation (texte)
 - note (optionnelle)
- Endpoint :


```
1 PUT /api/stagiaires/{id}/evaluation
2
```

Frontend – Angular

- Page Évaluation stagiaire

Détails

- Personne assignée: FY Facridine Ali Youssouf
- Priorité: Medium
- Parent: Aucun
- Date d'échéance: Aucun
- Étiquettes: Aucun

Figure 6 — Capture du ticket Jira FC-23 marqué comme « Done ».

V. Mission 2 — Dynamisation et gestion des Établissements (FC-24)

1. Contexte et expression du besoin

À l'issue de la mission FC-23, j'ai été affecté au ticket FC-24, dont le périmètre était sensiblement plus large. La plateforme contenait encore plusieurs zones où les informations relatives aux établissements étaient codées en dur dans les composants Angular ou directement dupliquées dans la table stagiaire, ce qui posait des problèmes évidents de cohérence et de maintenabilité. L'objectif était double : externaliser ces données dans une nouvelle table dédiée et exploiter cette nouvelle entité de bout en bout, depuis le schéma SQL jusqu'à l'interface utilisateur.

La mission a également intégré un volet UX : modernisation des fenêtres modales utilisées pour la création et l'édition des fiches, et introduction de styles SCSS dynamiques permettant de refléter visuellement la note attribuée à un stagiaire (badge coloré). Ce couplage entre travail de fond (modèle de données) et travail de surface (expérience utilisateur) est représentatif de la posture polyvalente attendue chez FenziConnect.

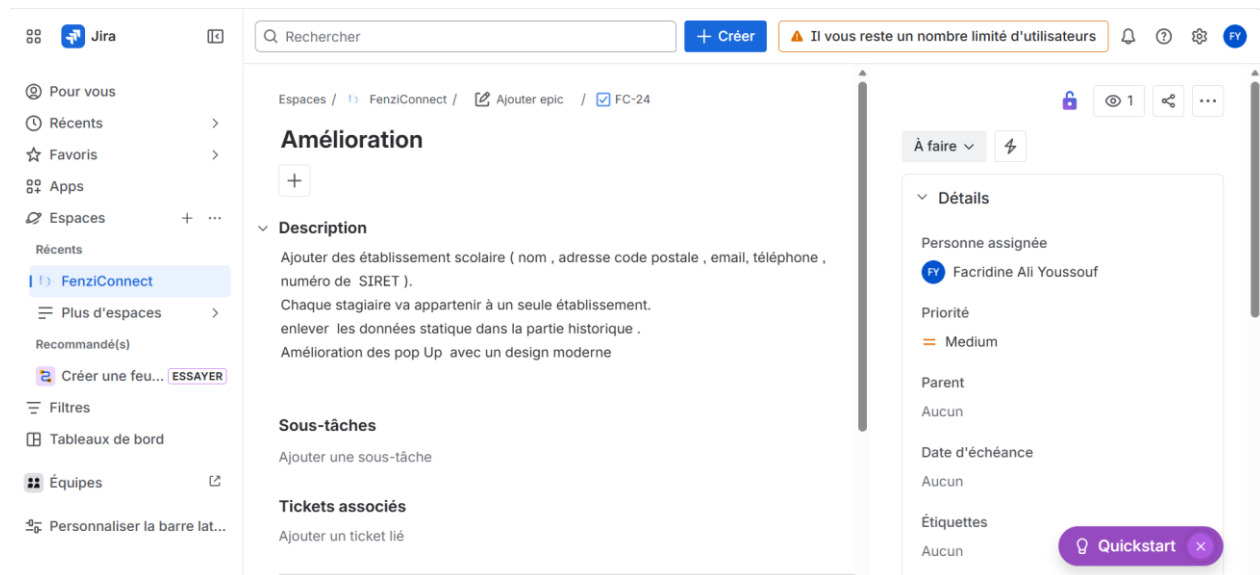


Figure 7 — Capture du ticket Jira FC-24 (description et critères d'acceptation).

2. Conception SQL et création de la table « établissement »

La première tâche technique consistait à concevoir la table etablissement et à la relier à la table stagiaire existante. Sept attributs ont été retenus en accord avec Madame FENZI : id, nom, adresse, codepostale, email, telephone et siret. La relation entre etablissement et stagiaire est de cardinalité 1-n : un établissement peut accueillir plusieurs stagiaires, mais un stagiaire est rattaché à un seul établissement. Cette relation est matérialisée par une clé étrangère etablissement_id ajoutée à la table stagiaire.

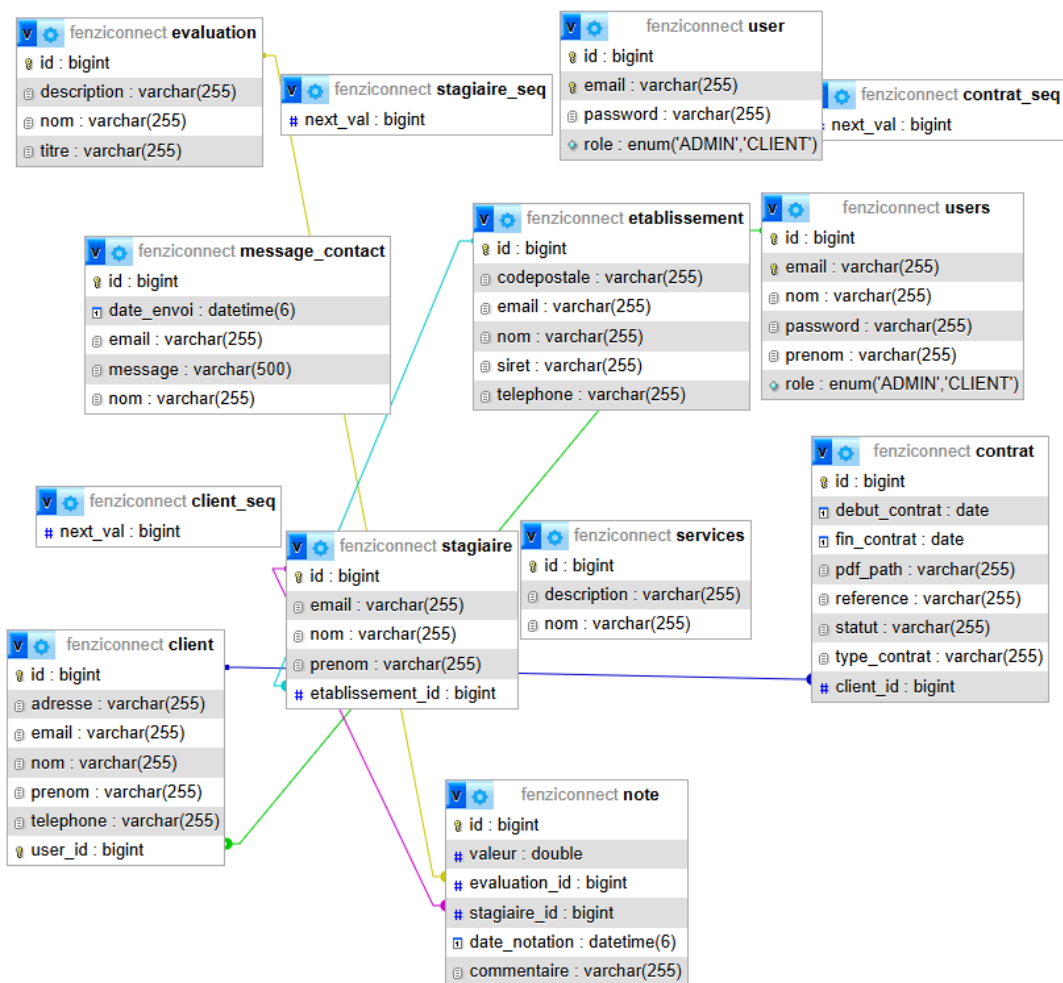


Figure 8 — Modèle Logique de Données (MLD) issu de PhpMyAdmin / dbdiagram.

Script SQL et contraintes d'intégrité référentielle

Le script de migration suivant a été exécuté sur la base de développement. La contrainte FOREIGN KEY garantit l'intégrité référentielle : il devient impossible d'associer un stagiaire à un établissement inexistant, et la suppression d'un établissement encore utilisé est bloquée par défaut (clause RESTRICT implicite). Cette rigueur, héritée des bonnes pratiques relationnelles, évite des incohérences difficiles à détecter une fois en production.

SQL — création de la table et de la clé étrangère

```
CREATE TABLE etablisement (
  id          BIGINT      NOT NULL AUTO_INCREMENT,
  nom         VARCHAR(150) NOT NULL,
  adresse     VARCHAR(255),
  codepostale VARCHAR(10),
  email       VARCHAR(150),
  telephone   VARCHAR(20),
  siret       VARCHAR(20),
```

```

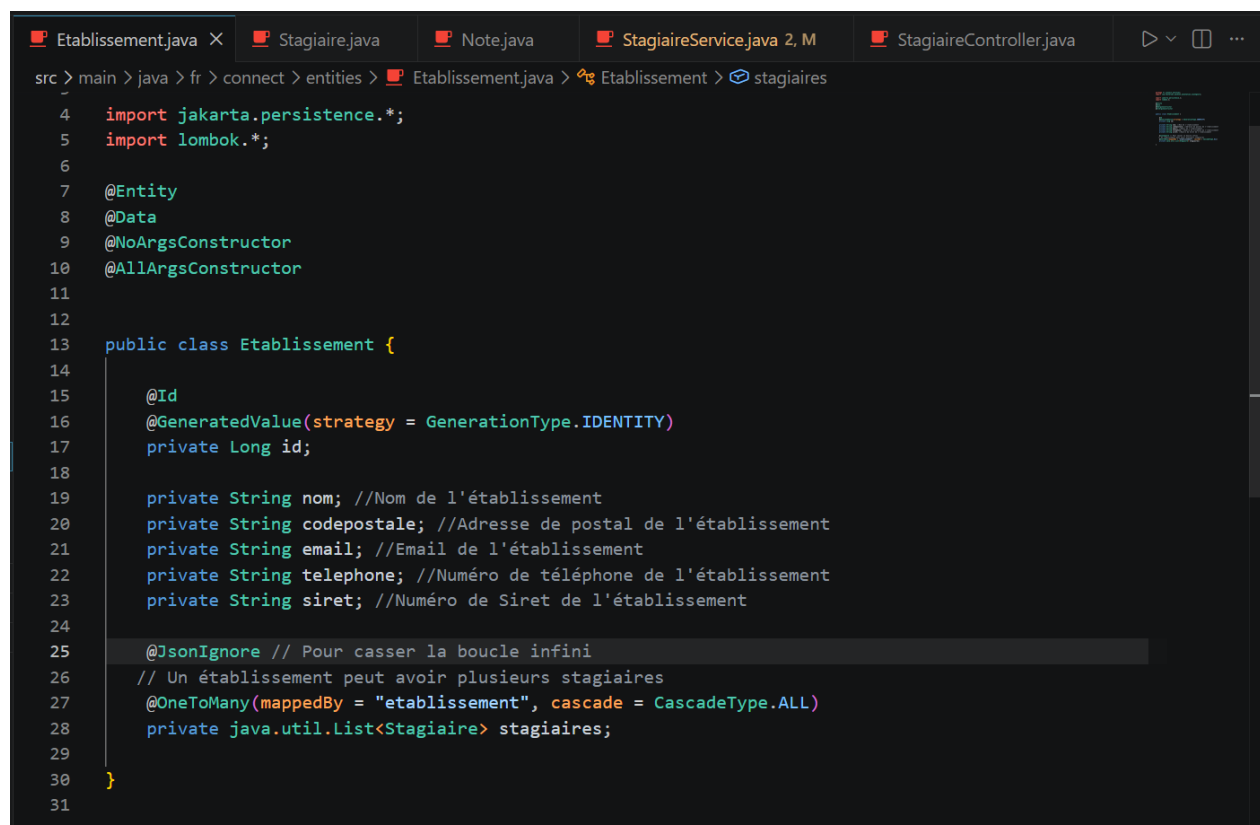
PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

ALTER TABLE stagiaire
ADD COLUMN etablisement_id BIGINT NULL,
ADD CONSTRAINT fk_stagiaire_etablisement
FOREIGN KEY (etablisement_id) REFERENCES etablisement(id);

```

3. Évolution du backend Spring Boot

Côté serveur, la prise en compte de la nouvelle table a nécessité trois évolutions concertées. D'abord, la création d'une entité JPA Etablissement et de son repository associé. Ensuite, l'ajout dans l'entité Stagiaire d'une relation @ManyToOne ciblant Etablissement, afin de matérialiser la cardinalité 1-n du modèle. Enfin, l'extension du contrôleur StagiaireController pour accepter, lors des créations et mises à jour, l'identifiant de l'établissement auquel le stagiaire est rattaché.



```

src > main > java > fr > connect > entites > Etablissement.java > Etablissement > stagiaires
4  import jakarta.persistence.*;
5  import lombok.*;
6
7  @Entity
8  @Data
9  @NoArgsConstructor
10 @AllArgsConstructor
11
12
13 public class Etablissement {
14
15     @Id
16     @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     private String nom; //Nom de l'établissement
20     private String codepostale; //Adresse de postal de l'établissement
21     private String email; //Email de l'établissement
22     private String telephone; //Numéro de téléphone de l'établissement
23     private String siret; //Numéro de Siret de l'établissement
24
25     @JsonIgnore // Pour casser la boucle infini
26     // Un établissement peut avoir plusieurs stagiaires
27     @OneToMany(mappedBy = "etablisement", cascade = CascadeType.ALL)
28     private java.util.List<Stagiaire> stagiaires;
29
30 }
31

```

Un nouveau contrôleur EtablissementController a été créé pour exposer les opérations CRUD de la nouvelle ressource. Une attention particulière a été portée à la configuration de SecurityConfig.java afin d'intégrer ces nouvelles routes au mécanisme d'autorisation : les opérations de modification (POST, PUT, DELETE) sont réservées aux profils administrateurs, tandis que les opérations de lecture (GET) restent accessibles aux profils consultation.

```

SecurityConfig.java
src > main > java > fr > connect > configuration > SecurityConfig.java > {} fr.connect.configuration
19 public class SecurityConfig {
20     @Bean
21     public SecurityFilterChain securityFilterChain(org.springframework.security.config.annotation.web
22
23         http
24         // * CORS moderne
25         .cors(cors -> {}) // utilisera le CorsConfigurationSource Bean
26         // * CSRF désactivé
27         .csrf(csrf -> csrf.disable())
28         // * Pas de session
29         .sessionManagement(sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
30         // * Sécurité des routes
31         .authorizeHttpRequests(auth -> auth
32             // routes publiques du collègue
33             .requestMatchers(...patterns: "/auth/signup", "/auth/login", "/api/stagiaires/**")
34             // routes publiques du dev (origin/dev)
35             .requestMatchers(...patterns: "/auth/**").permitAll()
36             // toutes les autres sécurisées
37             .anyRequest().authenticated()
38         )
39         // * JWT Filter avant UsernamePasswordAuthenticationFilter
40         .addFilterBefore(jwtAuthenticationFilter, beforeFilter: UsernamePasswordAuthenticationFil
41
42     return http.build();
43 }
44
45 // * Configuration CORS pour autoriser Angular localhost:4200
46 @Bean
47 public CorsConfigurationSource corsConfigurationSource() {
48     CorsConfiguration config = new CorsConfiguration();

```

Figure

9 — Extrait du fichier SecurityConfig.java configurant les rôles.

4. Refonte du frontend : tri, historique et UX

Le travail côté Angular a porté sur trois fonctions complémentaires : la suppression des données statiques, la dynamisation de l'historique et l'amélioration de l'ergonomie générale des écrans concernés. La fonction chargerHistoriqueReel() a notamment été réécrite pour consommer le nouvel endpoint, mettre en forme les données reçues et trier l'ensemble par date décroissante avant affichage.

```

48 chargerHistoriqueReel() {
49     this.noteService.getAllNotes().subscribe({
50         next: (donnees: any[]) => {
51             // On trie par date pour avoir les plus récentes en haut
52             this.historique = donnees
53                 .sort((a, b) => new Date(b.dateNotation).getTime() - new Date(a.dateNotation).getTime())
54                 .map(note => ({
55                     date: note.dateNotation ? new Date(note.dateNotation).toLocaleDateString() : 'Date inconnue',
56                     note: note.valeur,
57                     commentaire: note.commentaire,
58                     eleve: note.stagiaire ? `${note.stagiaire.prenom} ${note.stagiaire.nom}` : 'Inconnu',
59                     etablisement: note.stagiaire?.etablisement ? note.stagiaire.etablisement.nom : 'Non rattaché',
60                     module: note.evaluation ? note.evaluation.titre : 'Module inconnu',
61                     correcteur: 'Admin (Formateur)'
62                 }));
63         },
64         error: (err: any) => console.error('Erreur de chargement', err)
65     });
66 }

```

L'usage combiné de `.map()` et `.sort()` illustre une bonne pratique TypeScript : convertir d'abord les chaînes ISO en objets Date, puis trier de manière déterministe sur un timestamp numérique. Cette séparation des étapes facilite la lecture du code et évite les comparaisons lexicographiques trompeuses entre chaînes de caractères représentant des dates.

5. Modernisation des Modals et styles SCSS dynamiques

La dernière partie de la mission FC-24 a visé à harmoniser et moderniser les fenêtres modales utilisées pour créer ou modifier un établissement et un stagiaire. Les anciens Modals reposaient sur des composants hétérogènes hérités d'itérations précédentes du projet. Une feuille de styles SCSS commune a été introduite afin d'unifier les marges, les couleurs d'accent, les boutons d'action et l'animation d'ouverture, le tout en s'appuyant sur des variables centralisées pour faciliter une éventuelle évolution de charte graphique.

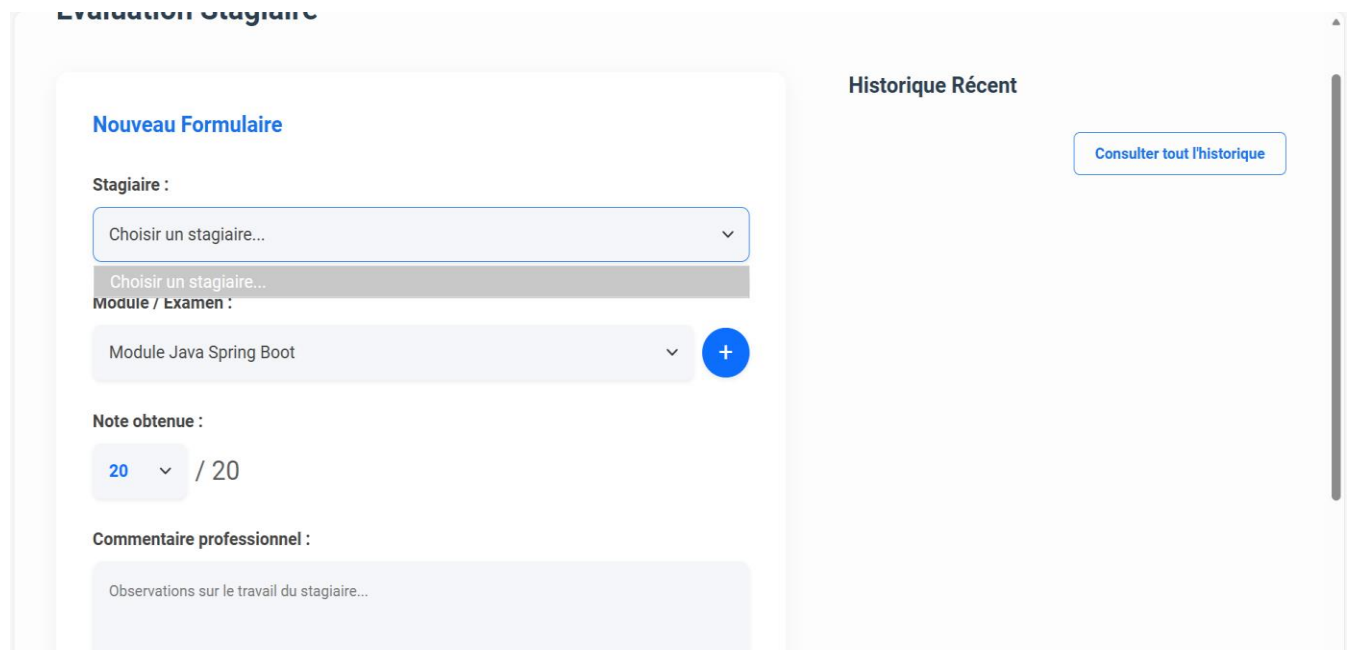


Figure 10 — Comparaison « avant / après » des Modals d'édition.

Badge de note dynamique

Pour rendre la consultation des évaluations plus immédiate, un badge de couleur a été ajouté à côté de chaque note : rouge pour les notes inférieures à 8, orange entre 8 et 13, vert au-delà. Cette logique est portée par une classe SCSS générée dynamiquement à partir de la valeur de la note dans le template Angular, ce qui évite l'usage de styles inline et conserve une séparation claire entre logique et présentation.

```
/* Couleurs dynamiques selon la note (Bordures de cartes) */  
.border-start-success { border-left-color: #198754 !important; } /* Vert : Note >= 15 */  
.border-start-warning { border-left-color: #ffc107 !important; } /* Jaune : Note entre 10 et 14 */  
.border-start-danger { border-left-color: #dc3545 !important; } /* Rouge : Note < 10 */
```

Figure 12 — Badges de notes dynamiques affichés dans la liste des stagiaires.

VI. Analyse comparative CACIB / FenziConnect

1. Cadre de la comparaison et méthode adoptée

Avoir effectué deux stages dans des structures aux profils diamétralement opposés constitue, à mes yeux, l'une des grandes richesses de mon parcours en BTS SIO SLAM. Le premier stage, effectué du 10 mai au 31 juillet 2025 (huit semaines) au sein du Crédit Agricole Corporate and Investment Bank (CACIB) sur le site de Montrouge (campus Evergreen), m'a confronté à l'univers des grandes directions informatiques bancaires : équipes étoffées et internationales, processus encadrés, environnements sécurisés, exigences de conformité fortes (RGPD, lutte contre le blanchiment, cybersécurité). Plus précisément, j'étais intégré à l'équipe Crossborder — Cash Management Services IT, en charge des paiements transfrontaliers et de la migration du standard SWIFT MT vers le standard MX (ISO 20022), sous la supervision de Monsieur Jérémie Konda, chef d'équipe.

Le second stage, effectué chez FenziConnect du 8 janvier au 20 février 2026 (six semaines), m'a au contraire plongé dans l'écosystème d'une auto-entreprise polyvalente, où la réactivité et l'autonomie priment sur la formalisation poussée des processus. Confronter explicitement ces deux expériences permet de tirer des enseignements précieux pour la suite de mon parcours.

La comparaison qui suit est volontairement structurée autour de deux axes choisis comme prioritaires : l'autonomie et la responsabilité confiées au stagiaire d'une part, l'apprentissage personnel rendu possible par chaque environnement d'autre part. Une lecture transversale de la stack technique et de la méthodologie viendra, en complément, étayer ces deux axes. Il ne s'agit pas de désigner un type de structure « meilleur » que l'autre, mais de mettre en évidence la complémentarité de ces deux expériences dans la construction d'une posture professionnelle équilibrée.

2. Méthodologie et processus de travail

Chez CACIB, le cycle de développement est très formalisé. Toute intervention, même mineure, s'inscrit dans un processus bien identifié : qualification du besoin, étude d'impact, validation technique, développement sur une branche dédiée, analyse automatique de la qualité du code via SonarQube, revue par un développeur senior, tests unitaires (JUnit 5 / Mockito), recette fonctionnelle, validation sécurité puis intégration via GitLab CI/CD. Cette rigueur trouve sa raison d'être dans le contexte bancaire : exigences réglementaires (RGPD, lutte contre le blanchiment d'argent, conformité interne), enjeux de stabilité d'un système d'information critique et sensibilité des données manipulées rendent indispensable un encadrement strict. Toute installation d'outil, même standard (Java SDK, Visual Studio Code, GitLab, SonarQube), passait par l'ouverture d'un ticket auprès du support technique et une validation hiérarchique.

Chez FenziConnect, la méthodologie est sensiblement plus légère. Le mode agile hybride pratiqué reprend les rituels essentiels de Scrum (planification, point quotidien, démonstration de

fin de sprint, rétrospective) sans en alourdir la mécanique. La revue de code reste systématique, mais les boucles de validation sont courtes et la mise en production peut être déclenchée plusieurs fois par sprint. L'installation des outils est immédiate, sans ticket préalable. Cette agilité serait inconcevable dans le contexte CACIB, mais elle est parfaitement adaptée à une structure servant directement ses clients TPE / PME, qui valorisent la réactivité et la souplesse. Aucune des deux approches n'est intrinsèquement supérieure : chacune est cohérente avec les contraintes de son contexte.

3. Comparaison des stacks techniques

Bien que partageant Java comme socle commun, les deux expériences m'ont confronté à des stacks et à des périmètres techniques très différents. Chez CACIB, j'ai été développeur Java pur, autour des messages SWIFT MT103 (puis MX / ISO 20022), avec utilisation intensive d'Apache POI pour la lecture des fichiers Excel et d'outils de qualité logicielle (SonarQube, JUnit 5, Mockito). Chez FenziConnect, le périmètre s'est élargi à un développement Full Stack complet, avec Spring Boot 3 côté serveur et Angular 16+ côté client. Le tableau ci-dessous synthétise ces différences.

Critère	CACIB (1er stage)	FenziConnect (2nd stage)
Périmètre	Backend Java pur	Full Stack (back + front)
Langages principaux	Java, SQL, formats SWIFT MT/MX	Java, TypeScript, SCSS, SQL
Frameworks / libs	Apache POI, JUnit 5, Mockito, PowerMockito	Spring Boot 3, Angular 16+, Hibernate, SweetAlert2
Qualité de code	SonarQube (analyse statique poussée)	Revue de code informelle entre pairs
Base de données	Bases internes sécurisées (paiements)	MySQL 8 (en local et recette)
UI / UX	Pas de contact direct	Forte exposition (Modals, SCSS, badges)
Outils projet	JIRA, GitLab, Microsoft Teams	Jira, Trello, GitLab, GitHub
Tests	Tests unitaires JUnit + Mockito + recette	Postman + recette croisée Front/Back
Cycle de release	Releases planifiées et encadrées (CI/CD)	Mises en production fréquentes
Langue de travail	Français + Anglais (équipe à Singapour)	Français

Cette comparaison met en évidence deux modèles d'ingénierie logicielle : un modèle « industriel » chez CACIB, où chaque composant est isolé, documenté, testé selon des standards internes et soumis à une analyse statique automatique avant fusion ; un modèle « artisanal » chez FenziConnect, fondé sur des outils open-source largement adoptés et sur une plus grande proximité avec l'ensemble de la chaîne logicielle. L'avantage d'avoir vécu les deux est de comprendre, concrètement, ce que chaque modèle apporte et coûte, plutôt que de s'en faire une représentation théorique.

4. Autonomie et responsabilité confiées

C'est sans doute sur ce critère que la différence entre les deux stages a été la plus marquée. Au sein de CACIB, j'étais intégré à l'équipe Crossborder, composée de plusieurs profils internationaux (développeurs, testeurs, chefs de projet), avec des collaborations directes vers Singapour. Mes missions (développement de l'outil de transformation Excel → MT103, analyse SonarQube, mise en place de tests unitaires Mockito) étaient encadrées par mon tuteur Monsieur Jérémie Konda et balisées par des spécifications précises. Toute proposition technique passait d'abord par une présentation à l'équipe pour validation, et tout commit devait être revu par un développeur senior avant intégration. Cette organisation présente l'immense avantage de sécuriser la production et de transmettre rapidement les bons réflexes à un stagiaire débutant, mais elle laisse, par construction, peu de place à des choix de conception personnels.

Chez FenziConnect, la situation s'est inversée. L'équipe étant très restreinte, j'ai été tenu pour responsable de l'intégralité des tickets FC-23 et FC-24 : analyse du besoin avec Madame FENZI, modélisation, choix d'implémentation, écriture du code backend et frontend, tests, démonstration en fin de sprint, puis correction des points soulevés. Cette autonomie est exigeante : elle suppose de savoir poser les bonnes questions au bon moment, d'anticiper les impacts d'une décision technique sur l'existant, et d'assumer la responsabilité d'un livrable directement utilisé par les utilisateurs finaux.

Au quotidien, cette différence d'autonomie change radicalement la manière de travailler. Chez CACIB, une journée type commençait par un daily meeting en anglais avec l'équipe internationale, puis se poursuivait par du développement Java ciblé (par exemple, ajout d'un mapping de champ MT103 ou correction d'une issue SonarQube). Le temps consacré à comprendre les conventions internes et les modules propriétaires était significatif. Chez FenziConnect, la même journée pouvait inclure une discussion produit le matin, une modification du modèle SQL en fin de matinée, le développement d'un endpoint Spring Boot l'après-midi et une démonstration en fin de journée. Cette polyvalence m'a appris à naviguer entre les couches techniques sans perdre de vue la vision d'ensemble.

5. Apprentissage personnel et complémentarité des deux stages

Au-delà des compétences techniques, ces deux expériences m'ont transmis des enseignements personnels très différents, et c'est précisément leur complémentarité qui en fait la valeur. Le stage chez CACIB m'a donné des bases solides en matière de rigueur : écrire un code qui respecte des standards stricts, documenter chaque modification, comprendre que la qualité d'un système dépend autant des conventions partagées que du talent individuel des développeurs. La mission de correction des erreurs critiques SonarQube — ajout systématique de blocs try-with-resources, vérification des entrées, gestion explicite des exceptions — a ancré

ces réflexes durablement. Cette discipline est devenue un automatisme que je conserve, même dans des environnements moins formalisés.

Le stage chez FenziConnect m’a, à l’inverse, appris à transformer cette rigueur en autonomie productive. Lorsqu’on est seul responsable d’un module, on ne peut pas se contenter d’appliquer des règles : il faut savoir hiérarchiser, faire des choix, les défendre auprès du tuteur, puis les remettre en cause si besoin. Cette posture est essentielle dès lors que l’on souhaite évoluer vers un poste de développeur confirmé. J’ai notamment compris à quel point la communication — claire, factuelle, brève — est un outil de travail à part entière, au même titre que les outils techniques.

Si je devais résumer en une phrase ce que ces deux stages m’apportent ensemble : CACIB m’a appris à écrire du code « propre » dans un cadre exigeant, en environnement bancaire international ; FenziConnect m’a appris à porter un module de bout en bout, sans filet et avec exigence envers moi-même. Aucune des deux compétences ne suffit isolément ; les deux combinées dessinent un profil de développeur autonome et fiable.

6. Synthèse comparative

Le tableau ci-dessous propose une lecture synthétique des enseignements que je retire de chacun des deux environnements. Il ne hiérarchise pas ces apports : il met en évidence leur caractère complémentaire pour un futur développeur Full Stack.

Critère	CACIB	FenziConnect
Durée	8 semaines	6 semaines
Taille de l’équipe	Grande (10+), internationale	Petite équipe polyvalente
Type de missions	MT103, SonarQube, Mockito	Modules Full Stack FC-23 et FC-24
Niveau d’autonomie	Limité (revue systématique)	Élevé (responsabilité du livrable)
Cycle décisionnel	Long, comités, validations	Court, échanges directs
Nature du code	Composant intégré à un système critique	Code livré directement au client
Apprentissage clé	Rigueur, qualité, conformité bancaire	Autonomie, polyvalence, communication
Posture attendue	Exécution précise, fiabilité	Initiative, responsabilité

7. Difficultés rencontrées et solutions apportées

Pour être honnête, le passage d’un environnement à l’autre n’a pas été immédiat. Les principales difficultés rencontrées chez FenziConnect, et les solutions que j’ai mises en place pour les surmonter, méritent d’être mentionnées.

- Changement de paradigme du backend pur (CACIB, traitement Java de fichiers Excel et messages SWIFT) au Full Stack (FenziConnect, Spring Boot + Angular) : la première semaine a été consacrée à monter en compétence sur Angular et TypeScript, langage que j’avais peu

pratiqué. La lecture systématique du guide officiel Angular et la reproduction de petits exercices m'ont permis de combler cette lacune.

- Architecture de dossiers complexe : le projet historique présentait plusieurs niveaux d'imbrication non documentés. J'ai pris le temps, dès la deuxième journée, de cartographier moi-même les dépendances entre modules, exercice qui m'a fait gagner beaucoup de temps par la suite.
- Gestion des permissions via SecurityConfig.java : la prise en main de Spring Security n'est pas triviale. J'ai abordé ce point en lisant la classe ligne par ligne avec Madame FENZI, puis en testant chaque modification par un appel Postman dédié, afin de vérifier le comportement attendu en cas d'authentification valide ou invalide.
- Passage de l'exécution stricte (revue automatique SonarQube + revue humaine systématique chez CACIB) à la prise d'initiative : il a fallu accepter de proposer des solutions qui pouvaient être discutées, voire rejetées, et apprendre à dépersonnaliser ces retours. J'ai progressivement intégré cette posture comme une opportunité d'amélioration plutôt qu'une remise en cause.

Avec le recul, ces difficultés ont été autant d'occasions d'apprentissage. Elles sont d'ailleurs étroitement corrélées aux compétences que j'ai le plus consolidées pendant le stage : autonomie, capacité d'adaptation et communication écrite avec un tuteur.

VII. Conclusion et perspectives professionnelles

1. Bilan général du stage

Ces six semaines passées chez FenziConnect, du 8 janvier au 20 février 2026, ont pleinement répondu aux objectifs que je m'étais fixés en début de parcours. Sur le plan technique, j'ai livré deux modules complets (FC-23 et FC-24) intégrant un périmètre Full Stack étendu : modèle de données, endpoints REST Spring Boot, composants Angular, gestion de la sécurité applicative et modernisation de l'expérience utilisateur. Sur le plan méthodologique, j'ai pris part au cycle agile complet de la structure et appris à porter la responsabilité de mes livrables auprès du client. Enfin, sur le plan personnel, j'ai consolidé ma confiance dans ma capacité à m'adapter à un environnement nouveau et à acquérir rapidement de nouvelles compétences.

Les deux modules développés sont aujourd'hui utilisés par les administrateurs de la plateforme. Cette mise en production constitue, pour un stagiaire, une forme de reconnaissance particulièrement gratifiante : elle signifie que le code écrit n'est pas resté un exercice scolaire, mais qu'il a apporté de la valeur à l'entreprise et à ses clients finaux.

2. Compétences techniques acquises ou consolidées

Les principales compétences techniques que ce stage m'a permis de consolider sont synthétisées ci-dessous. Elles s'ancrent directement dans le référentiel du BTS SIO SLAM tout en l'enrichissant d'outils professionnels concrets, complémentaires à ceux découverts chez CACIB.

Compétence	Mise en pratique pendant le stage
Conception orientée objet (Java)	Modélisation des entités Note et Etablissement, mise en place de relations JPA.
Bases de données relationnelles	Création de tables, contraintes d'intégrité référentielle, scripts de migration.
API REST	Endpoints PUT/GET/POST conformes aux conventions, gestion des codes HTTP.
Frontend Angular / TypeScript	Reactive Forms, services HttpClient, composants typés, RxJS de base.
Sécurité applicative	Lecture et adaptation de SecurityConfig.java, tests d'authentification.
UX / UI	Modernisation des Modals, badges SCSS dynamiques, alertes SweetAlert2.
Outillage projet	Git, GitLab, GitHub, Jira, Trello, Postman, Visual Studio Code.

3. Compétences transversales et soft skills

Au-delà du socle technique, ce stage a fortement contribué au développement de compétences transversales que j'estime indispensables à un futur développeur professionnel. Ces compétences viennent compléter celles acquises chez CACIB (travail en équipe internationale, communication en anglais lors des daily meetings, respect des règles de sécurité strictes).

- Autonomie : prise en charge complète d'un ticket, de l'analyse à la mise en production.
- Communication écrite : descriptions claires des merge requests, comptes rendus succincts en fin de sprint.
- Communication orale : présentation des avancées en démonstration de fin de sprint, capacité à argumenter un choix technique.
- Adaptabilité : passage rapide d'un stack pur Java à un stack Full Stack, prise en main d'un code existant.
- Esprit critique : remise en cause régulière de mes propres choix lors des revues, intégration constructive des retours.
- Sens de la responsabilité : conscience de l'impact d'un livrable sur les utilisateurs finaux et sur la réputation de l'entreprise.

4. Perspectives professionnelles à court terme

À l'issue du BTS SIO SLAM, j'envisage de poursuivre mes études en alternance dans une filière orientée développement Full Stack ou ingénierie logicielle. Le format de l'alternance présente, à mes yeux, un double avantage. Il permet d'abord de consolider les fondamentaux théoriques (architecture logicielle, design patterns, sécurité avancée, méthodes agiles) sur un cycle long, tout en restant ancré dans une pratique professionnelle continue. Il offre ensuite la possibilité d'éprouver, dans la durée, les deux types d'environnements que mes stages m'ont fait découvrir, afin d'affiner mon projet professionnel à la lumière d'une expérience plus longue qu'un simple stage.

Sur le plan technique, je souhaite approfondir trois axes en particulier : la maîtrise complète de Spring Boot et de son écosystème (Spring Security, Spring Cloud, tests d'intégration), le perfectionnement d'Angular avec une exploration plus poussée de RxJS et des architectures de gestion d'état, et enfin la découverte des problématiques DevOps (intégration continue, conteneurisation Docker, déploiements automatisés) qui constituent aujourd'hui un prolongement naturel des compétences de développeur, dans la lignée de l'expérience GitLab CI/CD amorcée chez CACIB.

5. Perspectives à moyen et long terme

À plus long terme, mon ambition est de devenir développeur Full Stack confirmé, capable d'intervenir avec aisance sur l'ensemble de la chaîne logicielle, de la base de données à l'interface utilisateur. Mes deux stages me confirment que ce profil polyvalent est non seulement réalisable, mais qu'il correspond à une demande forte du marché. Je reste ouvert à plusieurs types de structures pour la suite de ma carrière, en gardant en tête qu'un parcours équilibré bénéficie souvent d'expériences variées.

Une grande structure comme CACIB m'apportera probablement, à un moment donné, l'expertise approfondie sur un domaine précis (par exemple les paiements internationaux ou la

qualité logicielle outillée) et la rigueur des grands systèmes ; une structure plus petite continuera de cultiver l'autonomie, la polyvalence et la proximité avec les utilisateurs. Le choix entre les deux dépendra autant des opportunités que de l'étape de carrière dans laquelle je me trouverai. Ce que ces stages m'ont définitivement appris, c'est qu'il n'y a pas de réponse universelle : chaque environnement a ses richesses, et c'est en croisant les expériences que l'on construit un profil solide.

6. Mot de la fin

Ce stage de deuxième année chez FenziConnect restera une étape marquante de mon parcours, dans la continuité directe du stage effectué l'année précédente chez CACIB. Au-delà des livrables techniques, il m'a permis d'éprouver un rythme de travail professionnel, de valider mon projet d'orientation vers le développement Full Stack et de mesurer concrètement la complémentarité entre les enseignements théoriques du BTS SIO SLAM et la pratique en entreprise. Je tiens, une dernière fois, à exprimer ma reconnaissance à Madame Melissa FENZI pour son accompagnement, ainsi qu'à toutes les personnes qui ont rendu cette expérience possible — sans oublier Monsieur Jérémie Konda et l'équipe Crossborder de CACIB, qui m'avaient mis le pied à l'étrier l'année précédente.

C'est avec cette conviction renforcée et cette nouvelle base technique que j'aborde la suite de mon parcours, dans la continuité de ce qui a été initié pendant ces six semaines.